Data Mining: LIBSVM

Training, tuning, scaling, and evaluating SVM classifiers

Fall 2026 (7761)

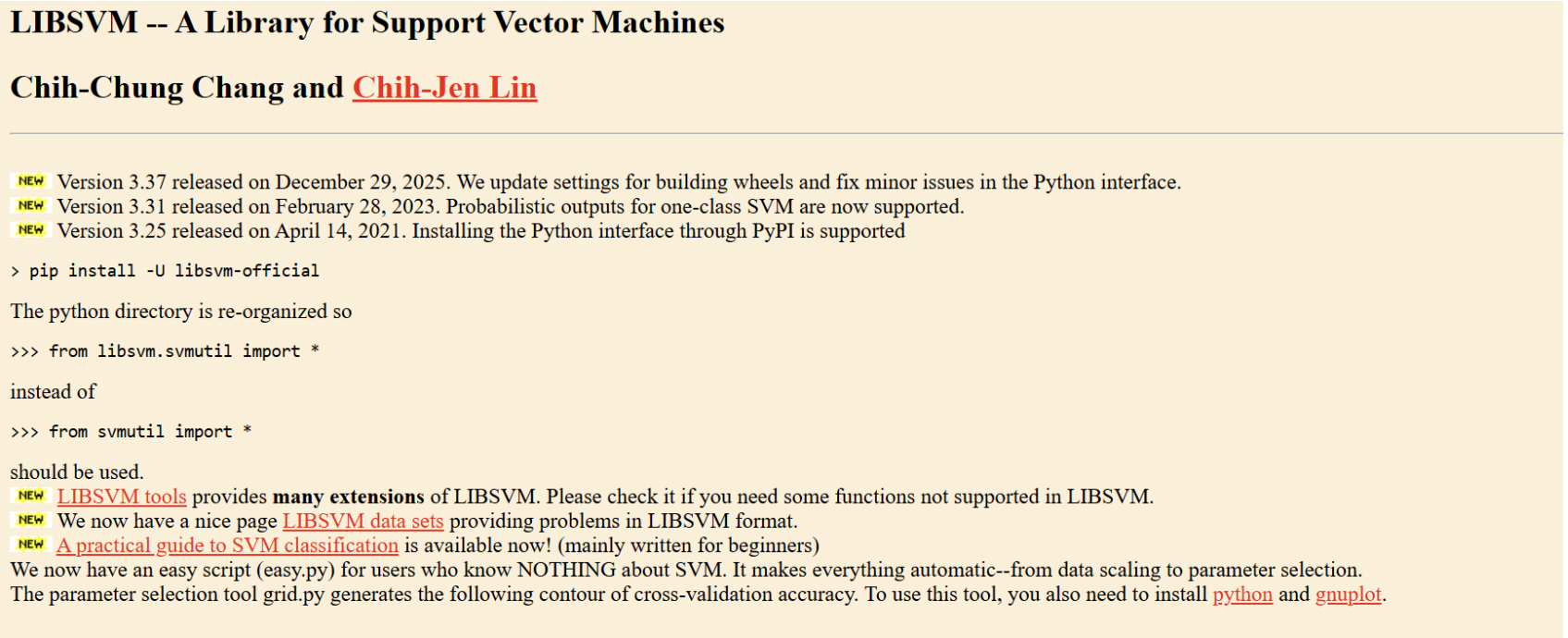
Dept. of Computer Science and Engineering

Instructor: Ting-Wei Lin

2026/11/13, Week 10

LIBSVM

- A command-line toolkit for support vector classification and regression.
  - http://www.csie.ntu.edu.tw/~cjlin/libsvm



LIBSVM -- A Library for Support Vector Machines

Chih-Chung Chang and Chih-Jen Lin

NEW Version 3.37 released on December 29, 2025. We update settings for building wheels and fix minor issues in the Python interface.
NEW Version 3.31 released on February 28, 2023. Probabilistic outputs for one-class SVM are now supported.
NEW Version 3.25 released on April 14, 2021. Installing the Python interface through PyPI is supported

```
> pip install -U libsvm-official
```

The python directory is re-organized so

```
>>> from libsvm.svmutil import *
```

instead of

```
>>> from svmutil import *
```

should be used.
NEW LIBSVM tools provides **many extensions** of LIBSVM. Please check it if you need some functions not supported in LIBSVM.
NEW We now have a nice page LIBSVM data sets providing problems in LIBSVM format.
NEW A practical guide to SVM classification is available now! (mainly written for beginners)
We now have an easy script (easy.py) for users who know NOTHING about SVM. It makes everything automatic--from data scaling to parameter selection.
The parameter selection tool grid.py generates the following contour of cross-validation accuracy. To use this tool, you also need to install python and gnuplot.

Download LIBSVM

Download LIBSVM

The current release (Version 3.37, December 2025) of **LIBSVM** can be obtained by downloading the zip file or tar.gz file. You can also check this github directory. Please e-mail us if you have problems to download the file.

The package includes the source code of the library in C++ and Java, and a simple program for scaling training data. A README file with detailed explanation is provided. For **MS Windows** users, there is a sub-directory in the zip file containing binary executable files. Precompiled Java class archive is also included.

Please read the COPYRIGHT notice before using **LIBSVM**.





Using SVM (SVM-Train)

```
Usage: svm-train [options] training_set_file [model_file]
options:
-s svm_type : set type of SVM (default 0)
    0 -- C-SVC      (multi-class classification)
    1 -- nu-SVC     (multi-class classification)
    2 -- one-class SVM
    3 -- epsilon-SVR    (regression)
    4 -- nu-SVR     (regression)
-t kernel_type : set type of kernel function (default 2)
    0 -- linear: u'*v
    1 -- polynomial: (gamma*u'*v + coef0)^degree
    2 -- radial basis function: exp(-gamma*|u-v|^2)
    3 -- sigmoid: tanh(gamma*u'*v + coef0)
    4 -- precomputed kernel (kernel values in training_set_file)
-d degree : set degree in kernel function (default 3)
-g gamma : set gamma in kernel function (default 1/num_features)
-r coef0 : set coef0 in kernel function (default 0)
-c cost : set the parameter C of C-SVC, epsilon-SVR, and nu-SVR (default 1)
-n nu : set the parameter nu of nu-SVC, one-class SVM, and nu-SVR (default 0.5)
-p epsilon : set the epsilon in loss function of epsilon-SVR (default 0.1)
-m cachesize : set cache memory size in MB (default 100)
-e epsilon : set tolerance of termination criterion (default 0.001)
-h shrinking : whether to use the shrinking heuristics, 0 or 1 (default 1)
-b probability_estimates : whether to train a model for probability estimates, 0 or 1 (default 0)
-wi weight : set the parameter C of class i to weight*C, for C-SVC (default 1)
-v n: n-fold cross validation mode
-q : quiet mode (no outputs)
```

```
svm-train.exe -t 2 -c 32 -g 0.00125 train model
```

Using SVM (SVM-Predict)

```
`svm-predict' Usage
==================
```

```
svm-predict.exe test model output.txt
```

```
Usage: svm-predict [options] test_file model_file output_file
options:
-b probability_estimates: whether to predict probability estimates, 0 or 1 (default 0).

model_file is the model file generated by svm-train.
test_file is the test data you want to predict.
svm-predict will produce output in the output_file.
```

The -b 1 option is required in both training and prediction. The output contains predicted labels and class probabilities.

Using SVM (SVM-scale)

```
`svm-scale' Usage
=================
```

```
svm-scale.exe -l 0 -u 1 -s scale.params train > train.scale
```

```
Usage: svm-scale [options] data_filename
options:
-l lower : x scaling lower limit (default -1)
-u upper : x scaling upper limit (default +1)
-y y_lower y_upper : y scaling limits (default: no y scaling)
-s save_filename : save scaling parameters to save_filename
-r restore_filename : restore scaling parameters from restore_filename
```

On Windows, in some cases svm-scale.exe output redirected with > can end up in a non-ASCII encoded file, and then svm-train cannot read it correctly

Training Procedure

- Source:
  - https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf

① Transform data to the format of

② Conduct simple scaling on the data

③ Consider the RBF kernel

④ Use cross-validation to find the best parameter C and γ

⑤ Use the best parameter C and γ to train the whole training set

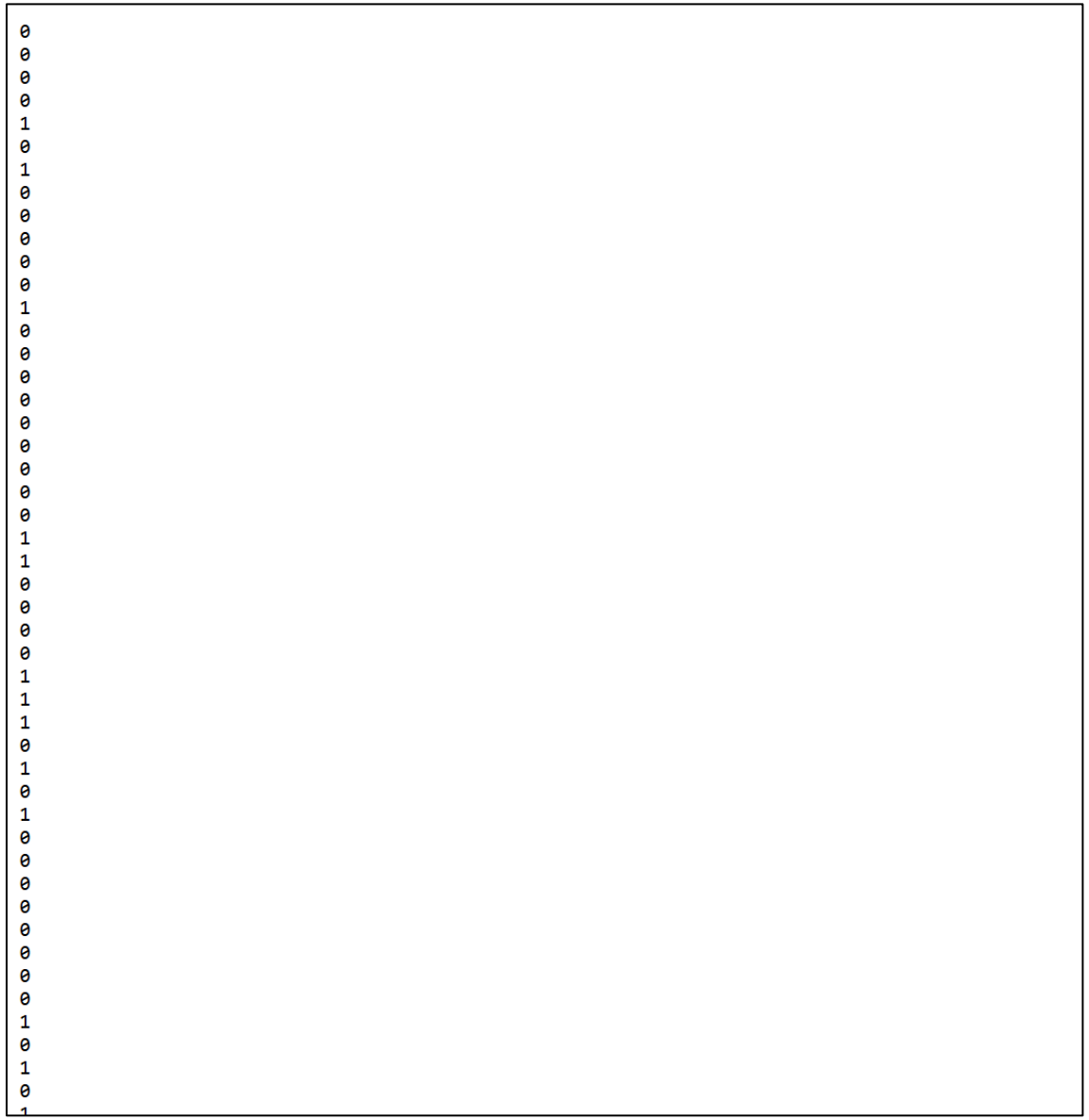
⑥ Test

Transform data to the format

- The format of training and testing data files is:
  - `<label> <index1>:<value1> <index2>:<value2> ...`
  - Each line contains an instance and is ended by a '\n' character
  - In the training set:
    - `<label>` is an integer indicating the class label (multi-class is supported)
  - In the test set:
    - `<label>` is used only to calculate accuracy or errors. If it's unknown, any number is fine

class label	feature 1	feature 2	feature 3	feature 4	feature 5
0	1:85.000000	2:92.000000	3:45.000000	4:27.000000	5:31.000000
0	1:88.000000	2:62.000000	3:20.000000	4:17.000000	5:9.000000
0	1:88.000000	2:67.000000	3:21.000000	4:11.000000	5:11.000000
0	1:92.000000	2:54.000000	3:22.000000	4:20.000000	5:7.000000
0	1:90.000000	2:60.000000	3:25.000000	4:19.000000	5:5.000000
0	1:89.000000	2:52.000000	3:13.000000	4:24.000000	5:15.000000
0	1:82.000000	2:62.000000	3:17.000000	4:17.000000	5:15.000000
1	1:89.000000	2:63.000000	3:22.000000	4:27.000000	5:10.000000
1	1:90.000000	2:84.000000	3:18.000000	4:23.000000	5:13.000000
1	1:88.000000	2:95.000000	3:25.000000	4:19.000000	5:14.000000
1	1:89.000000	2:35.000000	3:27.000000	4:29.000000	5:17.000000
1	1:91.000000	2:80.000000	3:37.000000	4:23.000000	5:27.000000
1	1:91.000000	2:109.000000	3:33.000000	4:15.000000	5:18.000000
1	1:91.000000	2:65.000000	3:17.000000	4:5.000000	5:7.000000
1	1:92.000000	2:66.000000	3:21.000000	4:16.000000	5:33.000000

Predict


0
0
0
0
1
0
1
0
0
0
0
0
0
1
0
0
0
0
0
0
0
0
0
0
0
1
1
0
0
0
0
1
1
1
0
1
0
1
0
0
0
0
0
0
0
0
0
1
0
1
0
1

Restore scale on test data

- Do not let values from the test set affect the preprocessing rule.
- Scale on training data:
  - `svm-scale.exe -l 0 -u 1 -s scale.params train > train.scale`
- Restore scale on test data
  - `svm-scale.exe -r scale.params test > test.scale`

Categorical Features: use one-hot encoding

- LIBSVM accepts numeric feature values, so convert category membership into binary indicators.
  - Color = {red, blue, green}
    - One-hot encode as: red ([1, 0, 0]), blue ([0, 1, 0]), green ([0, 0, 1])
  - Red with positive label ⇒ 1 1:1 2:0 3:0
  - Blue with negative label ⇒ 0 1:0 2:1 3:0
  - Green with positive label ⇒ 1 1:0 2:0 3:1

Optional: Sparse LIBSVM format

- For high-dimensional one-hot features, write only the non-zero entries.
  - Dense format:
    - Feature vector: [0, 0, 1, 0, 0, 3.5], positive
    - 1 1:0 2:0 3:1 4:0 5:0 6:3.5
  - Sparse LIBSVM format:
    - Feature vector: [0, 0, 1, 0, 0, 3.5], positive
    - 1 3:1 6:3.5